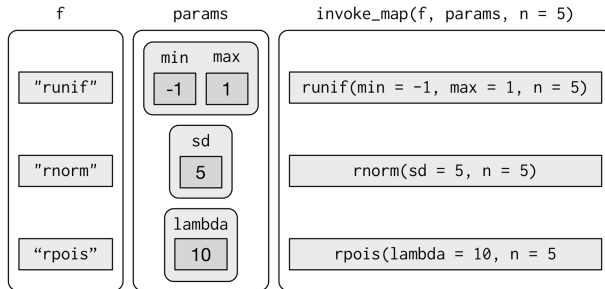


To handle this case, you can use `invoke_map()`:

```
invoke_map(f, param, n = 5) %>% str()
#> List of 3
#> $ : num [1:5] 0.762 0.36 -0.714 0.531 0.254
#> $ : num [1:5] 3.07 -3.09 1.1 5.64 9.07
#> $ : int [1:5] 9 14 8 9 7
```



The first argument is a list of functions or a character vector of function names. The second argument is a list of lists giving the arguments that vary for each function. The subsequent arguments are passed on to every function.

And again, you can use `tribble()` to make creating these matching pairs a little easier:

```
sim <- tribble(
  ~f,      ~params,
  "runif", list(min = -1, max = 1),
  "rnorm", list(sd = 5),
  "rpois", list(lambda = 10)
)
sim %>%
  mutate(sim = invoke_map(f, params, n = 10))
```

Walk

Walk is an alternative to map that you use when you want to call a function for its side effects, rather than for its return value. You typically do this because you want to render output to the screen or save files to disk—the important thing is the action, not the return value. Here's a very simple example:

```
x <- list(1, "a", 3)

x %>%
```

```
walk(print)
#> [1] 1
#> [1] "a"
#> [1] 3
```

`walk()` is generally not that useful compared to `walk2()` or `pwalk()`. For example, if you had a list of plots and a vector of filenames, you could use `pwalk()` to save each file to the corresponding location on disk:

```
library(ggplot2)
plots <- mtcars %>%
  split(.$cyl) %>%
  map(~ggplot(., aes(mpg, wt)) + geom_point())
paths <- stringr::str_c(names(plots), ".pdf")

pwalk(list(paths, plots), ggsave, path = tempdir())
```

`walk()`, `walk2()`, and `pwalk()` all invisibly return `.x`, the first argument. This makes them suitable for use in the middle of pipelines.

Other Patterns of For Loops

purrr provides a number of other functions that abstract over other types of for loops. You'll use them less frequently than the `map` functions, but they're useful to know about. The goal here is to briefly illustrate each function, so hopefully it will come to mind if you see a similar problem in the future. Then you can go look up the documentation for more details.

Predicate Functions

A number of functions work with *predicate* functions that return either a single TRUE or FALSE.

`keep()` and `discard()` keep elements of the input where the predicate is TRUE or FALSE, respectively:

```
iris %>%
  keep(is.factor) %>%
  str()
#> 'data.frame': 150 obs. of 1 variable:
#> $ Species: Factor w/ 3 levels "setosa","versicolor",...: ...

iris %>%
  discard(is.factor) %>%
  str()
#> 'data.frame': 150 obs. of 4 variables:
```

```
#> $ Sepal.Length: num 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
#> $ Sepal.Width : num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3 ...
#> $ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 ...
#> $ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 ...
```

`some()` and `every()` determine if the predicate is true for any or for all of the elements:

```
x <- list(1:5, letters, list(10))

x %>%
  some(is_character)
#> [1] TRUE

x %>%
  every(is_vector)
#> [1] TRUE
```

`detect()` finds the first element where the predicate is true; `detect_index()` returns its position:

```
x <- sample(10)
x
#> [1] 8 7 5 6 9 2 10 1 3 4

x %>%
  detect(~ . > 5)
#> [1] 8

x %>%
  detect_index(~ . > 5)
#> [1] 1
```

`head_while()` and `tail_while()` take elements from the start or end of a vector while a predicate is true:

```
x %>%
  head_while(~ . > 5)
#> [1] 8 7

x %>%
  tail_while(~ . > 5)
#> integer(0)
```

Reduce and Accumulate

Sometimes you have a complex list that you want to reduce to a simple list by repeatedly applying a function that reduces a pair to a singleton. This is useful if you want to apply a two-table **dplyr** verb to multiple tables. For example, you might have a list of data frames,

and you want to reduce to a single data frame by joining the elements together:

```
dfs <- list(
  age = tibble(name = "John", age = 30),
  sex = tibble(name = c("John", "Mary"), sex = c("M", "F")),
  trt = tibble(name = "Mary", treatment = "A")
)

dfs %>% reduce(full_join)
#> Joining, by = "name"
#> Joining, by = "name"
#> # A tibble: 2 × 4
#>   name    age  sex treatment
#>   <chr> <dbl> <chr>    <chr>
#> 1 John    30    M      <NA>
#> 2 Mary    NA    F      A
```

Or maybe you have a list of vectors, and want to find the intersection:

```
vs <- list(
  c(1, 3, 5, 6, 10),
  c(1, 2, 3, 7, 8, 10),
  c(1, 2, 3, 4, 8, 9, 10)
)

vs %>% reduce(intersect)
#> [1] 1 3 10
```

The `reduce` function takes a “binary” function (i.e., a function with two primary inputs), and applies it repeatedly to a list until there is only a single element left.

`Accumulate` is similar but it keeps all the interim results. You could use it to implement a cumulative sum:

```
x <- sample(10)
x
#> [1] 6 9 8 5 2 4 7 1 10 3
x %>% accumulate(`+`)
#> [1] 6 15 23 28 30 34 41 42 52 55
```

Exercises

1. Implement your own version of `every()` using a for loop. Compare it with `purrr::every()`. What does **purrr**’s version do that your version doesn’t?

2. Create an enhanced `col_sum()` that applies a summary function to every numeric column in a data frame.
3. A possible base R equivalent of `col_sum()` is:

```
col_sum3 <- function(df, f) {  
  is_num <- sapply(df, is.numeric)  
  df_num <- df[, is_num]  
  
  sapply(df_num, f)  
}
```

But it has a number of bugs as illustrated with the following inputs:

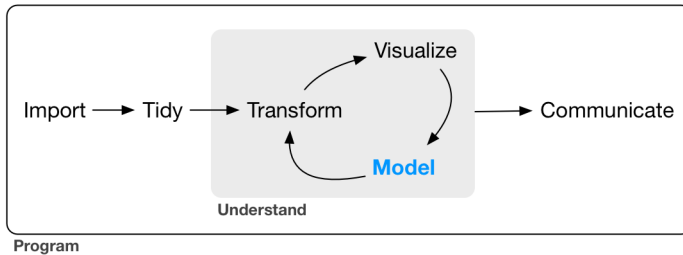
```
df <- tibble(  
  x = 1:3,  
  y = 3:1,  
  z = c("a", "b", "c")  
)  
# OK  
col_sum3(df, mean)  
# Has problems: don't always return numeric vector  
col_sum3(df[1:2], mean)  
col_sum3(df[1], mean)  
col_sum3(df[0], mean)
```

What causes the bugs?

PART IV

Model

Now that you are equipped with powerful programming tools we can finally return to modeling. You'll use your new tools of data wrangling and programming to fit many models and understand how they work. The focus of this book is on exploration, not confirmation or formal inference. But you'll learn a few basic tools that help you understand the variation within your models.



The goal of a model is to provide a simple low-dimensional summary of a dataset. Ideally, the model will capture true “signals” (i.e., patterns generated by the phenomenon of interest), and ignore “noise” (i.e., random variation that you’re not interested in). Here we only cover “predictive” models, which, as the name suggests, generate predictions. There is another type of model that we’re not going to discuss: “data discovery” models. These models don’t make pre-

dictions, but instead help you discover interesting relationships within your data. (These two categories of models are sometimes called supervised and unsupervised, but I don't think that terminology is particularly illuminating.)

This book is not going to give you a deep understanding of the mathematical theory that underlies models. It will, however, build your intuition about how statistical models work, and give you a family of useful tools that allow you to use models to better understand your data:

- In **Chapter 18**, you'll learn how models work mechanistically, focusing on the important family of linear models. You'll learn general tools for gaining insight into what a predictive model tells you about your data, focusing on simple simulated datasets.
- In **Chapter 19**, you'll learn how to use models to pull out known patterns in real data. Once you have recognized an important pattern it's useful to make it explicit in a model, because then you can more easily see the subtler signals that remain.
- In **Chapter 20**, you'll learn how to use many simple models to help understand complex datasets. This is a powerful technique, but to access it you'll need to combine modeling and programming tools.

These topics are notable because of what they don't include: any tools for quantitatively assessing models. That is deliberate: precisely quantifying a model requires a couple of big ideas that we just don't have the space to cover here. For now, you'll rely on qualitative assessment and your natural skepticism. In **"Learning More About Models"** on page 396, we'll point you to other resources where you can learn more.

Hypothesis Generation Versus Hypothesis Confirmation

In this book, we are going to use models as a tool for exploration, completing the trifecta of the tools for EDA that were introduced in **Part I**. This is not how models are usually taught, but as you will see, models are an important tool for exploration. Traditionally, the focus of modeling is on inference, or for confirming that a hypothesis is true. Doing this correctly is not complicated, but it is hard.

There is a pair of ideas that you must understand in order to do inference correctly:

- Each observation can either be used for exploration or confirmation, not both.
- You can use an observation as many times as you like for exploration, but you can only use it once for confirmation. As soon as you use an observation twice, you've switched from confirmation to exploration.

This is necessary because to confirm a hypothesis you must use data independent of the data that you used to generate the hypothesis. Otherwise you will be overoptimistic. There is absolutely nothing wrong with exploration, but you should never sell an exploratory analysis as a confirmatory analysis because it is fundamentally misleading.

If you are serious about doing a confirmatory analysis, one approach is to split your data into three pieces before you begin the analysis:

- 60% of your data goes into a *training* (or exploration) set. You're allowed to do anything you like with this data: visualize it and fit tons of models to it.
- 20% goes into a *query* set. You can use this data to compare models or visualizations by hand, but you're not allowed to use it as part of an automated process.
- 20% is held back for a *test* set. You can only use this data ONCE, to test your final model.

This partitioning allows you to explore the training data, occasionally generating candidate hypotheses that you check with the query set. When you are confident you have the right model, you can check it once with the test data.

(Note that even when doing confirmatory modeling, you will still need to do EDA. If you don't do any EDA you will remain blind to the quality problems with your data.)

Model Basics with modelr

Introduction

The goal of a model is to provide a simple low-dimensional summary of a dataset. In the context of this book we're going to use models to partition data into patterns and residuals. Strong patterns will hide subtler trends, so we'll use models to help peel back layers of structure as we explore a dataset.

However, before we can start using models on interesting, real datasets, you need to understand the basics of how models work. For that reason, this chapter of the book is unique because it uses only simulated datasets. These datasets are very simple, and not at all interesting, but they will help you understand the essence of modeling before you apply the same techniques to real data in the next chapter.

There are two parts to a model:

1. First, you define a *family of models* that express a precise, but generic, pattern that you want to capture. For example, the pattern might be a straight line, or a quadratic curve. You will express the model family as an equation like $y = a_1 * x + a_2$ or $y = a_1 * x ^ a_2$. Here, x and y are known variables from your data, and a_1 and a_2 are parameters that can vary to capture different patterns.
2. Next, you generate a *fitted model* by finding the model from the family that is the closest to your data. This takes the generic

model family and makes it specific, like $y = 3 * x + 7$ or $y = 9 * x ^ 2$.

It's important to understand that a fitted model is just the closest model from a family of models. That implies that you have the “best” model (according to some criteria); it doesn't imply that you have a good model and it certainly doesn't imply that the model is “true.” George Box puts this well in his famous aphorism:

All models are wrong, but some are useful.

It's worth reading the fuller context of the quote:

Now it would be very remarkable if any system existing in the real world could be exactly represented by any simple model. However, cunningly chosen parsimonious models often do provide remarkably useful approximations. For example, the law $PV = RT$ relating pressure P , volume V and temperature T of an “ideal” gas via a constant R is not exactly true for any real gas, but it frequently provides a useful approximation and furthermore its structure is informative since it springs from a physical view of the behavior of gas molecules.

For such a model there is no need to ask the question “Is the model true?” If “truth” is to be the “whole truth” the answer must be “No.” The only question of interest is “Is the model illuminating and useful?”

The goal of a model is not to uncover truth, but to discover a simple approximation that is still useful.

Prerequisites

In this chapter we'll use the **modelr** package, which wraps around base R's modeling functions to make them work naturally in a pipe.

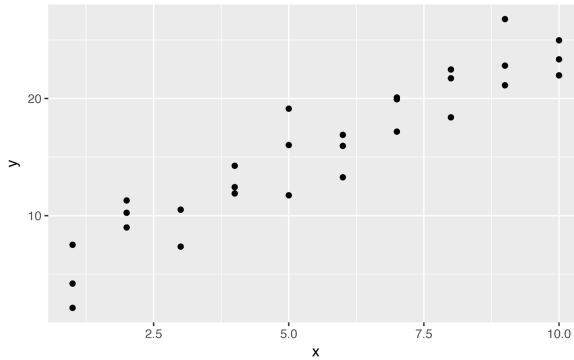
```
library(tidyverse)

library(modelr)
options(na.action = na.warn)
```

A Simple Model

Let's take a look at the simulated dataset `sim1`. It contains two continuous variables, `x` and `y`. Let's plot them to see how they're related:

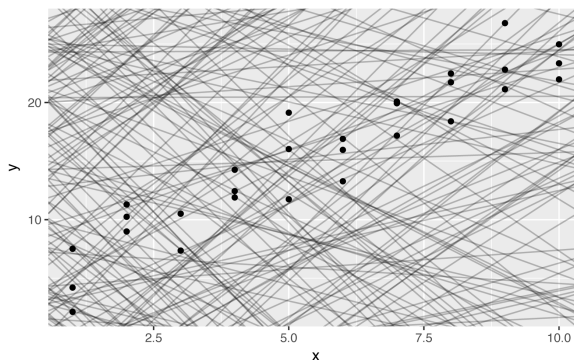
```
ggplot(sim1, aes(x, y)) +
  geom_point()
```



You can see a strong pattern in the data. Let's use a model to capture that pattern and make it explicit. It's our job to supply the basic form of the model. In this case, the relationship looks linear, i.e., $y = a_0 + a_1 * x$. Let's start by getting a feel for what models from that family look like by randomly generating a few and overlaying them on the data. For this simple case, we can use `geom_abline()`, which takes a slope and intercept as parameters. Later on we'll learn more general techniques that work with any model:

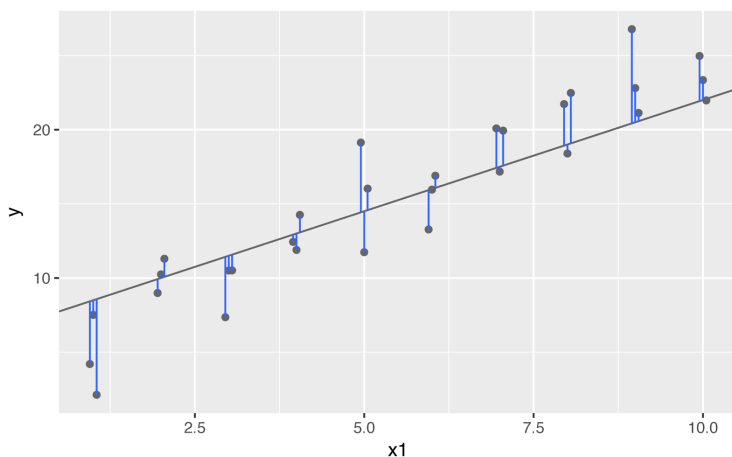
```
models <- tibble(
  a1 = runif(250, -20, 40),
  a2 = runif(250, -5, 5)
)

ggplot(sim1, aes(x, y)) +
  geom_abline(
    aes(intercept = a1, slope = a2),
    data = models, alpha = 1/4
  ) +
  geom_point()
```



There are 250 models on this plot, but a lot are really bad! We need to find the good models by making precise our intuition that a good model is “close” to the data. We need a way to quantify the distance between the data and a model. Then we can fit the model by finding the values of a_0 and a_1 that generate the model with the smallest distance from this data.

One easy place to start is to find the vertical distance between each point and the model, as in the following diagram. (Note that I’ve shifted the x values slightly so you can see the individual distances.)



This distance is just the difference between the y value given by the model (the *prediction*), and the actual y value in the data (the *response*).

To compute this distance, we first turn our model family into an R function. This takes the model parameters and the data as inputs, and gives values predicted by the model as output:

```
model1 <- function(a, data) {
  a[1] + data$x * a[2]
}
model1(c(7, 1.5), sim1)
#> [1] 8.5 8.5 8.5 10.0 10.0 10.0 11.5 11.5 11.5 13.0 13.0
#> [12] 13.0 14.5 14.5 14.5 16.0 16.0 16.0 17.5 17.5 17.5 19.0
#> [23] 19.0 19.0 20.5 20.5 20.5 22.0 22.0 22.0
```

Next, we need some way to compute an overall distance between the predicted and actual values. In other words, the plot shows 30 distances: how do we collapse that into a single number?

One common way to do this in statistics is to use the “root-mean-squared deviation.” We compute the difference between actual and predicted, square them, average them, and then take the square root. This distance has lots of appealing mathematical properties, which we’re not going to talk about here. You’ll just have to take my word for it!

```
measure_distance <- function(mod, data) {
  diff <- data$y - model1(mod, data)
  sqrt(mean(diff ^ 2))
}
measure_distance(c(7, 1.5), sim1)
#> [1] 2.67
```

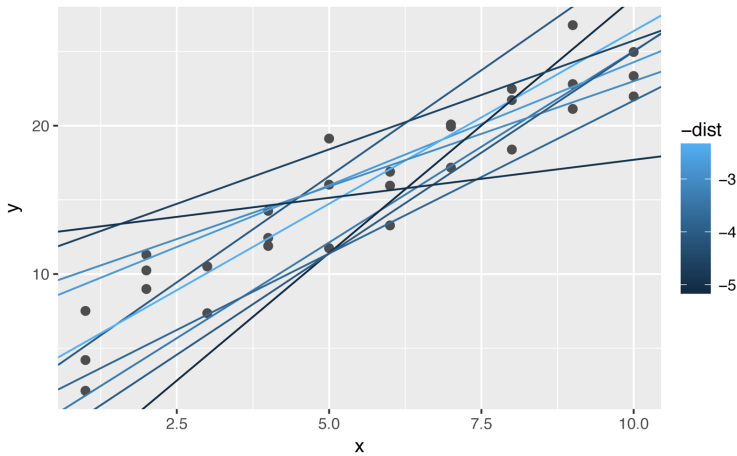
Now we can use **purrr** to compute the distance for all the models defined previously. We need a helper function because our distance function expects the model as a numeric vector of length 2:

```
sim1_dist <- function(a1, a2) {
  measure_distance(c(a1, a2), sim1)
}

models <- models %>%
  mutate(dist = purrr::map2_dbl(a1, a2, sim1_dist))
models
#> # A tibble: 250 × 3
#>       a1      a2  dist
#>   <dbl> <dbl> <dbl>
#> 1 -15.15  0.0889  30.8
#> 2  30.06 -0.8274  13.2
#> 3  16.05  2.2695  13.2
#> 4 -10.57  1.3769  18.7
#> 5 -19.56 -1.0359  41.8
#> 6   7.98  4.5948  19.3
#> # ... with 244 more rows
```

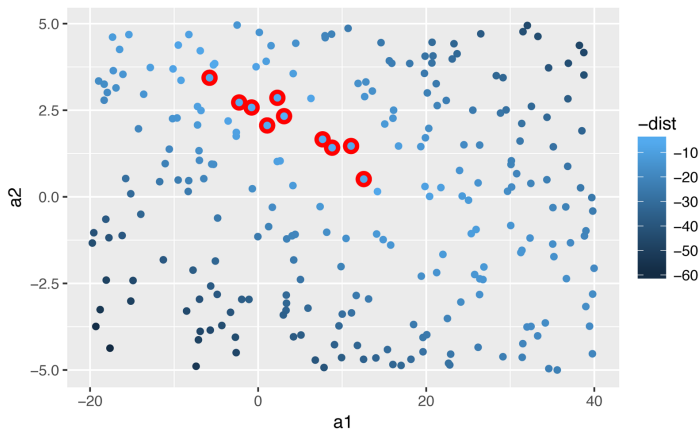
Next, let’s overlay the 10 best models on to the data. I’ve colored the models by `-dist`: this is an easy way to make sure that the best models (i.e., the ones with the smallest distance) get the brightest colors:

```
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, color = "grey30") +
  geom_abline(
    aes(intercept = a1, slope = a2, color = -dist),
    data = filter(models, rank(dist) <= 10)
  )
```



We can also think about these models as observations, and visualize them with a scatterplot of `a1` versus `a2`, again colored by `-dist`. We can no longer directly see how the model compares to the data, but we can see many models at once. Again, I've highlighted the 10 best models, this time by drawing red circles underneath them:

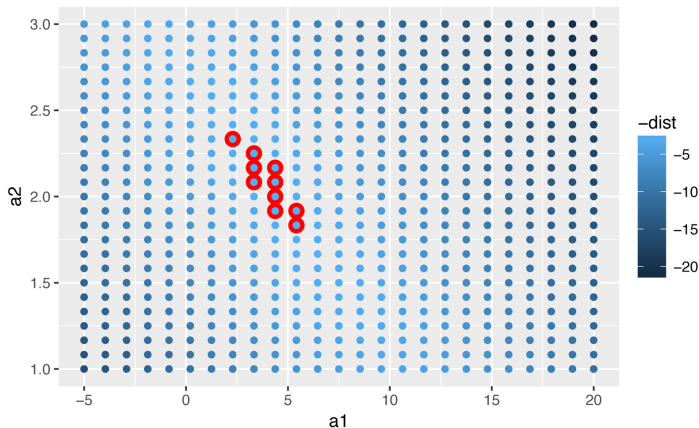
```
ggplot(models, aes(a1, a2)) +
  geom_point(
    data = filter(models, rank(dist) <= 10),
    size = 4, color = "red"
  ) +
  geom_point(aes(colour = -dist))
```



Instead of trying lots of random models, we could be more systematic and generate an evenly spaced grid of points (this is called a grid search). I picked the parameters of the grid roughly by looking at where the best models were in the preceding plot:

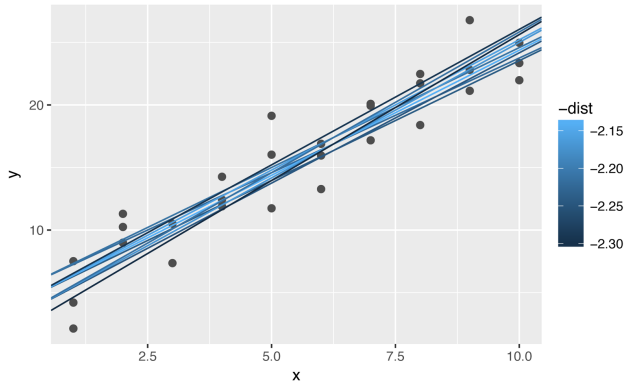
```
grid <- expand_grid(
  a1 = seq(-5, 20, length = 25),
  a2 = seq(1, 3, length = 25)
) %>%
  mutate(dist = purrr::map2_dbl(a1, a2, sim1_dist))

grid %>%
  ggplot(aes(a1, a2)) +
  geom_point(
    data = filter(grid, rank(dist) <= 10),
    size = 4, colour = "red"
  ) +
  geom_point(aes(color = -dist))
```



When you overlay the best 10 models back on the original data, they all look pretty good:

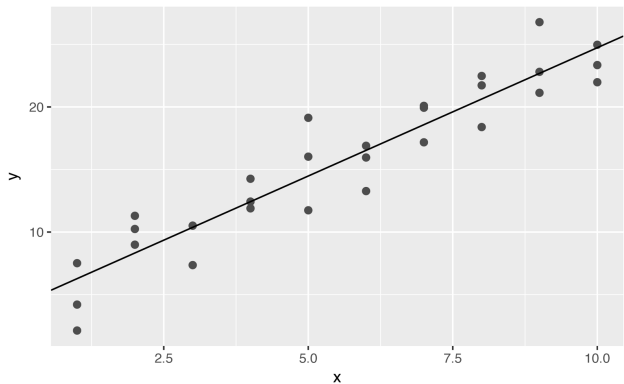
```
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, color = "grey30") +
  geom_abline(
    aes(intercept = a1, slope = a2, color = -dist),
    data = filter(grid, rank(dist) <= 10)
  )
```



You could imagine iteratively making the grid finer and finer until you narrowed in on the best model. But there's a better way to tackle that problem: a numerical minimization tool called Newton–Raphson search. The intuition of Newton–Raphson is pretty simple: you pick a starting point and look around for the steepest slope. You then ski down that slope a little way, and then repeat again and again, until you can't go any lower. In R, we can do that with `optim()`:

```
best <- optim(c(0, 0), measure_distance, data = sim1)
best$par
#> [1] 4.22 2.05

ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, color = "grey30") +
  geom_abline(intercept = best$par[1], slope = best$par[2])
```



Don't worry too much about the details of how `optim()` works. It's the intuition that's important here. If you have a function that defines the distance between a model and a dataset, and an algorithm that can minimize that distance by modifying the parameters of the model, you can find the best model. The neat thing about this approach is that it will work for any family of models that you can write an equation for.

There's one more approach that we can use for this model, because it is a special case of a broader family: linear models. A linear model has the general form $y = a_1 + a_2 * x_1 + a_3 * x_2 + \dots + a_n * x_{(n - 1)}$. So this simple model is equivalent to a general linear model where n is 2 and x_1 is x . R has a tool specifically designed for fitting linear models called `lm()`. `lm()` has a special way to specify the model family: formulas. Formulas look like $y \sim x$, which `lm()` will translate to a function like $y = a_1 + a_2 * x$. We can fit the model and look at the output:

```
sim1_mod <- lm(y ~ x, data = sim1)
coef(sim1_mod)
#> (Intercept)          x
#>      4.22      2.05
```

These are exactly the same values we got with `optim()`! Behind the scenes `lm()` doesn't use `optim()` but instead takes advantage of the mathematical structure of linear models. Using some connections between geometry, calculus, and linear algebra, `lm()` actually finds the closest model in a single step, using a sophisticated algorithm. This approach is faster and guarantees that there is a global minimum.

Exercises

1. One downside of the linear model is that it is sensitive to unusual values because the distance incorporates a squared term. Fit a linear model to the following simulated data, and visualize the results. Rerun a few times to generate different simulated datasets. What do you notice about the model?

```
sim1a <- tibble(
  x = rep(1:10, each = 3),
  y = x * 1.5 + 6 + rt(length(x), df = 2)
)
```

2. One way to make linear models more robust is to use a different distance measure. For example, instead of root-mean-squared distance, you could use mean-absolute distance:

```
measure_distance <- function(mod, data) {  
  diff <- data$y - make_prediction(mod, data)  
  mean(abs(diff))  
}
```

Use `optim()` to fit this model to the previous simulated data and compare it to the linear model.

3. One challenge with performing numerical optimization is that it's only guaranteed to find one local optima. What's the problem with optimizing a three-parameter model like this?

```
model1 <- function(a, data) {  
  a[1] + data$x * a[2] + a[3]  
}
```

Visualizing Models

For simple models, like the one in the previous section, you can figure out what pattern the model captures by carefully studying the model family and the fitted coefficients. And if you ever take a statistics course on modeling, you're likely to spend a lot of time doing just that. Here, however, we're going to take a different tack. We're going to focus on understanding a model by looking at its predictions. This has a big advantage: every type of predictive model makes predictions (otherwise what use would it be?) so we can use the same set of techniques to understand any type of predictive model.

It's also useful to see what the model doesn't capture, the so-called residuals that are left after subtracting the predictions from the data. Residuals are powerful because they allow us to use models to remove striking patterns so we can study the subtler trends that remain.

Predictions

To visualize the predictions from a model, we start by generating an evenly spaced grid of values that covers the region where our data lies. The easiest way to do that is to use `modelr::data_grid()`. Its

first argument is a data frame, and for each subsequent argument it finds the unique variables and then generates all combinations:

```
grid <- sim1 %>%  
  data_grid(x)  
grid  
#> # A tibble: 10 × 1  
#>       x  
#>   <int>  
#> 1     1  
#> 2     2  
#> 3     3  
#> 4     4  
#> 5     5  
#> 6     6  
#> # ... with 4 more rows
```

(This will get more interesting when we start to add more variables to our model.)

Next we add predictions. We'll use `modelr::add_predictions()`, which takes a data frame and a model. It adds the predictions from the model to a new column in the data frame:

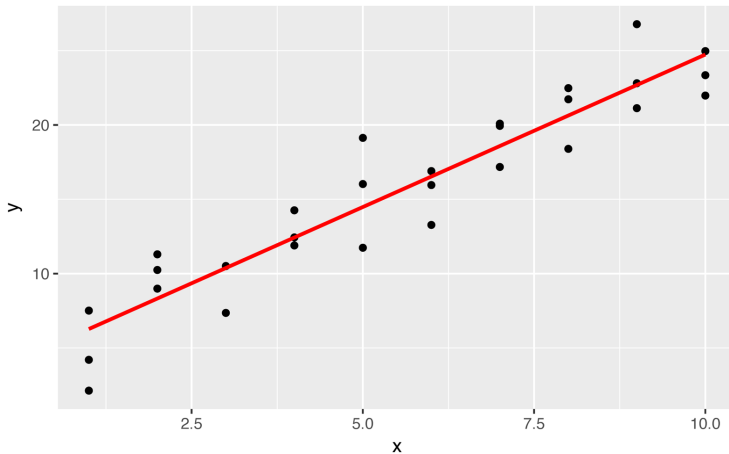
```
grid <- grid %>%  
  add_predictions(sim1_mod)  
grid  
#> # A tibble: 10 × 2  
#>       x pred  
#>   <int> <dbl>  
#> 1     1  6.27  
#> 2     2  8.32  
#> 3     3 10.38  
#> 4     4 12.43  
#> 5     5 14.48  
#> 6     6 16.53  
#> # ... with 4 more rows
```

(You can also use this function to add predictions to your original dataset.)

Next, we plot the predictions. You might wonder about all this extra work compared to just using `geom_abline()`. But the advantage of this approach is that it will work with *any* model in R, from the simplest to the most complex. You're only limited by your visualization skills. For more ideas about how to visualize more complex model types, you might try <http://vita.had.co.nz/papers/model-vis.html>.

```
ggplot(sim1, aes(x)) +  
  geom_point(aes(y = y)) +
```

```
geom_line(
  aes(y = pred),
  data = grid,
  colour = "red",
  size = 1
)
```



Residuals

The flip side of predictions are *residuals*. The predictions tell you the pattern that the model has captured, and the residuals tell you what the model has missed. The residuals are just the distances between the observed and predicted values that we computed earlier.

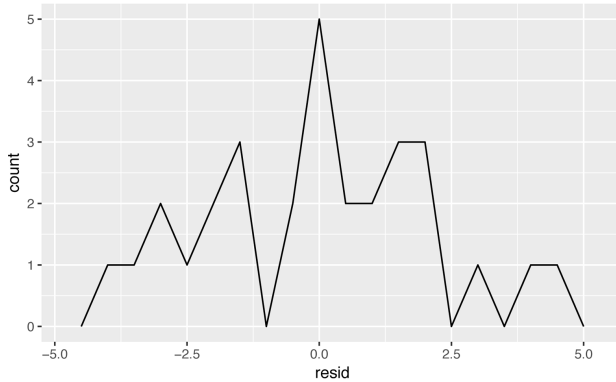
We add residuals to the data with `add_residuals()`, which works much like `add_predictions()`. Note, however, that we use the original dataset, not a manufactured grid. This is because to compute residuals we need actual `y` values:

```
sim1 <- sim1 %>%
  add_residuals(sim1_mod)
sim1
#> # A tibble: 30 × 3
#>       x     y resid
#>   <int> <dbl> <dbl>
#> 1     1  4.20 -2.072
#> 2     1  7.51  1.238
#> 3     1  2.13 -4.147
#> 4     2  8.99  0.665
#> 5     2 10.24  1.919
```

```
#> 6      2 11.30  2.973
#> # ... with 24 more rows
```

There are a few different ways to understand what the residuals tell us about the model. One way is to simply draw a frequency polygon to help us understand the spread of the residuals:

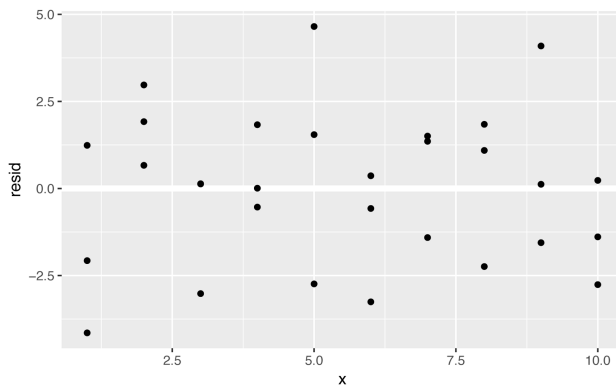
```
ggplot(sim1, aes(resid)) +
  geom_freqpoly(binwidth = 0.5)
```



This helps you calibrate the quality of the model: how far away are the predictions from the observed values? Note that the average of the residual will always be 0.

You'll often want to re-create plots using the residuals instead of the original predictor. You'll see a lot of that in the next chapter:

```
ggplot(sim1, aes(x, resid)) +
  geom_ref_line(h = 0) +
  geom_point()
```



This looks like random noise, suggesting that our model has done a good job of capturing the patterns in the dataset.

Exercises

1. Instead of using `lm()` to fit a straight line, you can use `loess()` to fit a smooth curve. Repeat the process of model fitting, grid generation, predictions, and visualization on `sim1` using `loess()` instead of `lm()`. How does the result compare to `geom_smooth()`?
2. `add_predictions()` is paired with `gather_predictions()` and `spread_predictions()`. How do these three functions differ?
3. What does `geom_ref_line()` do? What package does it come from? Why is displaying a reference line in plots showing residuals useful and important?
4. Why might you want to look at a frequency polygon of absolute residuals? What are the pros and cons compared to looking at the raw residuals?

Formulas and Model Families

You've seen formulas before when using `facet_wrap()` and `facet_grid()`. In R, formulas provide a general way of getting "special behavior." Rather than evaluating the values of the variables right away, they capture them so they can be interpreted by the function.

The majority of modeling functions in R use a standard conversion from formulas to functions. You've seen one simple conversion already: $y \sim x$ is translated to $y = a_1 + a_2 * x$. If you want to see what R actually does, you can use the `model_matrix()` function. It takes a data frame and a formula and returns a tibble that defines the model equation: each column in the output is associated with one coefficient in the model, and the function is always $y = a_1 * out1 + a_2 * out2$. For the simplest case of $y \sim x1$ this shows us something interesting:

```
df <- tribble(
  ~y, ~x1, ~x2,
  4, 2, 5,
  5, 1, 6
```



```
)
model_matrix(df, y ~ x1)
#> # A tibble: 2 × 2
#>   `(Intercept)`    x1
#>   <dbl> <dbl>
#> 1         1     2
#> 2         1     1
```

The way that R adds the intercept to the model is just by having a column that is full of ones. By default, R will always add this column. If you don't want that, you need to explicitly drop it with `-1`:

```
model_matrix(df, y ~ x1 - 1)
#> # A tibble: 2 × 1
#>   x1
#>   <dbl>
#> 1     2
#> 2     1
```

The model matrix grows in an unsurprising way when you add more variables to the model:

```
model_matrix(df, y ~ x1 + x2)
#> # A tibble: 2 × 3
#>   `(Intercept)`    x1    x2
#>   <dbl> <dbl> <dbl>
#> 1         1     2     5
#> 2         1     1     6
```

This formula notation is sometimes called “Wilkinson-Rogers notation,” and was initially described in *Symbolic Description of Factorial Models for Analysis of Variance*, by G. N. Wilkinson and C. E. Rogers. It's worth digging up and reading the original paper if you'd like to understand the full details of the modeling algebra.

The following sections expand on how this formula notation works for categorical variables, interactions, and transformation.

Categorical Variables

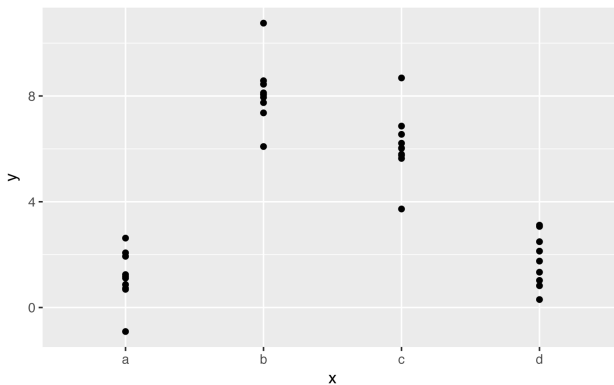
Generating a function from a formula is straightforward when the predictor is continuous, but things get a bit more complicated when the predictor is categorical. Imagine you have a formula like `y ~ sex`, where `sex` could either be male or female. It doesn't make sense to convert that to a formula like `y = x_0 + x_1 * sex` because `sex` isn't a number—you can't multiply it! Instead what R does is convert it to `y = x_0 + x_1 * sex_male` where `sex_male` is one if `sex` is male and zero otherwise:

```
df <- tribble(
  ~ sex, ~ response,
  "male", 1,
  "female", 2,
  "male", 1
)
model_matrix(df, response ~ sex)
#> # A tibble: 3 × 2
#>   `(Intercept)` sexmale
#>   <dbl>      <dbl>
#> 1         1         1
#> 2         1         0
#> 3         1         1
```

You might wonder why R also doesn't create a `sexfemale` column. The problem is that would create a column that is perfectly predictable based on the other columns (i.e., `sexfemale = 1 - sexmale`). Unfortunately the exact details of why this is a problem is beyond the scope of this book, but basically it creates a model family that is too flexible, and will have infinitely many models that are equally close to the data.

Fortunately, however, if you focus on visualizing predictions you don't need to worry about the exact parameterization. Let's look at some data and models to make that concrete. Here's the `sim2` dataset from **modelr**:

```
ggplot(sim2) +
  geom_point(aes(x, y))
```



We can fit a model to it, and generate predictions:

```
mod2 <- lm(y ~ x, data = sim2)
```

```

grid <- sim2 %>%
  data_grid(x) %>%
  add_predictions(mod2)
grid
#> # A tibble: 4 × 2
#>       x   pred
#>   <chr> <dbl>
#> 1     a  1.15
#> 2     b  8.12
#> 3     c  6.13
#> 4     d  1.91

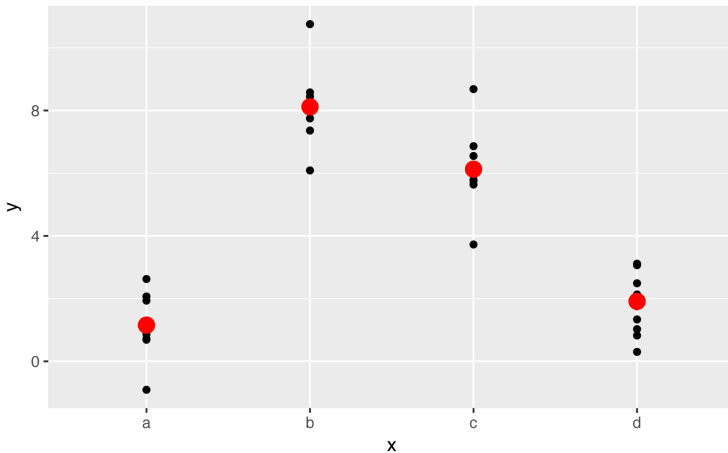
```

Effectively, a model with a categorical x will predict the mean value for each category. (Why? Because the mean minimizes the root-mean-squared distance.) That's easy to see if we overlay the predictions on top of the original data:

```

ggplot(sim2, aes(x)) +
  geom_point(aes(y = y)) +
  geom_point(
    data = grid,
    aes(y = pred),
    color = "red",
    size = 4
  )

```



You can't make predictions about levels that you didn't observe. Sometimes you'll do this by accident so it's good to recognize this error message:

```

tibble(x = "e") %>%
  add_predictions(mod2)

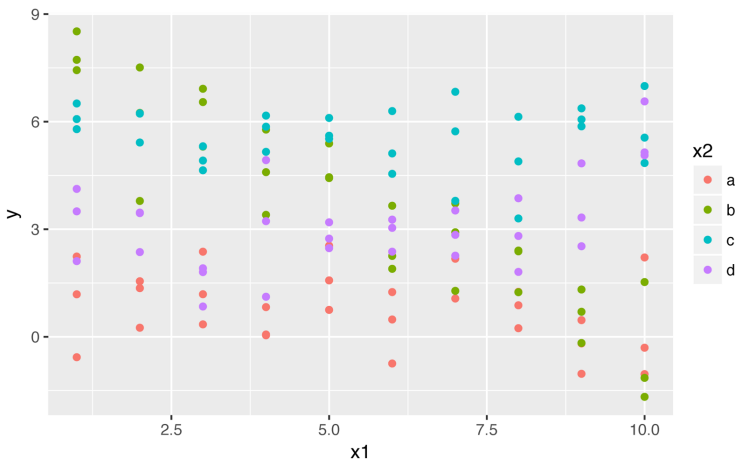
```

```
#> Error in model.frame.default(Terms, newdata, na.action =
#> na.action, xlev = object$xlevels): factor x has new level e
```

Interactions (Continuous and Categorical)

What happens when you combine a continuous and a categorical variable? `sim3` contains a categorical predictor and a continuous predictor. We can visualize it with a simple plot:

```
ggplot(sim3, aes(x1, y)) +
  geom_point(aes(color = x2))
```



There are two possible models you could fit to this data:

```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3)
```

When you add variables with `+`, the model will estimate each effect independent of all the others. It's possible to fit the so-called interaction by using `*`. For example, `y ~ x1 * x2` is translated to $y = a_0 + a_1 * a1 + a_2 * a2 + a_{12} * a1 * a2$. Note that whenever you use `*`, both the interaction and the individual components are included in the model.

To visualize these models we need two new tricks:

- We have two predictors, so we need to give `data_grid()` both variables. It finds all the unique values of `x1` and `x2` and then generates all combinations.

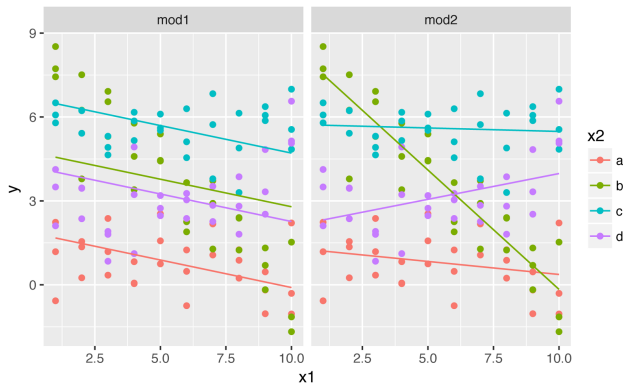
- To generate predictions from both models simultaneously, we can use `gather_predictions()`, which adds each prediction as a row. The complement of `gather_predictions()` is `spread_predictions()`, which adds each prediction to a new column.

Together this gives us:

```
grid <- sim3 %>%
  data_grid(x1, x2) %>%
  gather_predictions(mod1, mod2)
grid
#> # A tibble: 80 × 4
#>   model  x1    x2  pred
#>   <chr> <int> <fctr> <dbl>
#> 1 mod1     1    a  1.67
#> 2 mod1     1    b  4.56
#> 3 mod1     1    c  6.48
#> 4 mod1     1    d  4.03
#> 5 mod1     2    a  1.48
#> 6 mod1     2    b  4.37
#> # ... with 74 more rows
```

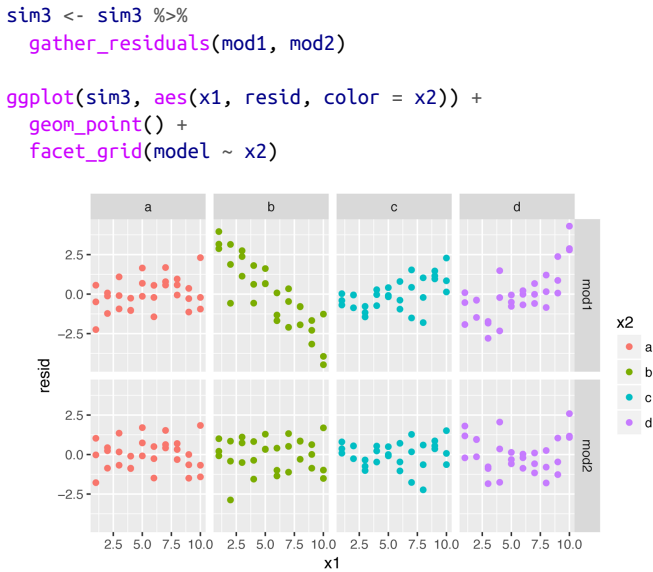
We can visualize the results for both models on one plot using facetting:

```
ggplot(sim3, aes(x1, y, color = x2)) +
  geom_point() +
  geom_line(data = grid, aes(y = pred)) +
  facet_wrap(~ model)
```



Note that the model that uses `+` has the same slope for each line, but different intercepts. The model that uses `*` has a different slope and intercept for each line.

Which model is better for this data? We can take look at the residuals. Here I've faceted by both model and x2 because it makes it easier to see the pattern within each group:



There is little obvious pattern in the residuals for `mod2`. The residuals for `mod1` show that the model has clearly missed some pattern in b, and less so, but still present, is pattern in c, and d. You might wonder if there's a precise way to tell which of `mod1` or `mod2` is better. There is, but it requires a lot of mathematical background, and we don't really care. Here, we're interested in a qualitative assessment of whether or not the model has captured the pattern that we're interested in.

Interactions (Two Continuous)

Let's take a look at the equivalent model for two continuous variables. Initially things proceed almost identically to the previous example:

```
mod1 <- lm(y ~ x1 + x2, data = sim4)
mod2 <- lm(y ~ x1 * x2, data = sim4)

grid <- sim4 %>%
  data_grid(
    x1 = seq_range(x1, 5),
```

```

x2 = seq_range(x2, 5)
) %>%
gather_predictions(mod1, mod2)
grid
#> # A tibble: 50 × 4
#>   model    x1    x2    pred
#>   <chr> <dbl> <dbl> <dbl>
#> 1 mod1 -1.0 -1.0  0.996
#> 2 mod1 -1.0 -0.5 -0.395
#> 3 mod1 -1.0  0.0 -1.786
#> 4 mod1 -1.0  0.5 -3.177
#> 5 mod1 -1.0  1.0 -4.569
#> 6 mod1 -0.5 -1.0  1.907
#> # ... with 44 more rows

```

Note my use of `seq_range()` inside `data_grid()`. Instead of using every unique value of `x`, I'm going to use a regularly spaced grid of five values between the minimum and maximum numbers. It's probably not super important here, but it's a useful technique in general. There are three other useful arguments to `seq_range()`:

- `pretty = TRUE` will generate a “pretty” sequence, i.e., something that looks nice to the human eye. This is useful if you want to produce tables of output:

```

seq_range(c(0.0123, 0.923423), n = 5)
#> [1] 0.0123 0.2401 0.4679 0.6956 0.9234
seq_range(c(0.0123, 0.923423), n = 5, pretty = TRUE)
#> [1] 0.0 0.2 0.4 0.6 0.8 1.0

```

- `trim = 0.1` will trim off 10% of the tail values. This is useful if the variable has a long-tailed distribution and you want to focus on generating values near the center:

```

x1 <- rcauchy(100)
seq_range(x1, n = 5)
#> [1] -115.9 -83.5 -51.2 -18.8 13.5
seq_range(x1, n = 5, trim = 0.10)
#> [1] -13.84 -8.71 -3.58 1.55 6.68
seq_range(x1, n = 5, trim = 0.25)
#> [1] -2.1735 -1.0594 0.0547 1.1687 2.2828
seq_range(x1, n = 5, trim = 0.50)
#> [1] -0.725 -0.268 0.189 0.647 1.104

```

- `expand = 0.1` is in some sense the opposite of `trim()`; it expands the range by 10%:

```

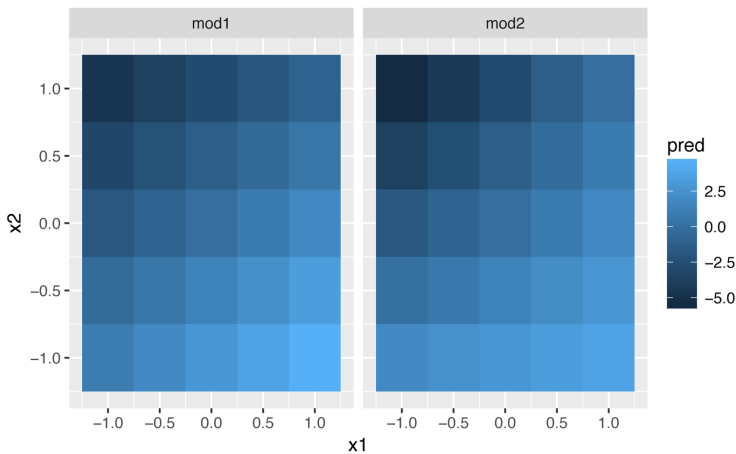
x2 <- c(0, 1)
seq_range(x2, n = 5)
#> [1] 0.00 0.25 0.50 0.75 1.00

```

```
seq_range(x2, n = 5, expand = 0.10)
#> [1] -0.050  0.225  0.500  0.775  1.050
seq_range(x2, n = 5, expand = 0.25)
#> [1] -0.125  0.188  0.500  0.812  1.125
seq_range(x2, n = 5, expand = 0.50)
#> [1] -0.250  0.125  0.500  0.875  1.250
```

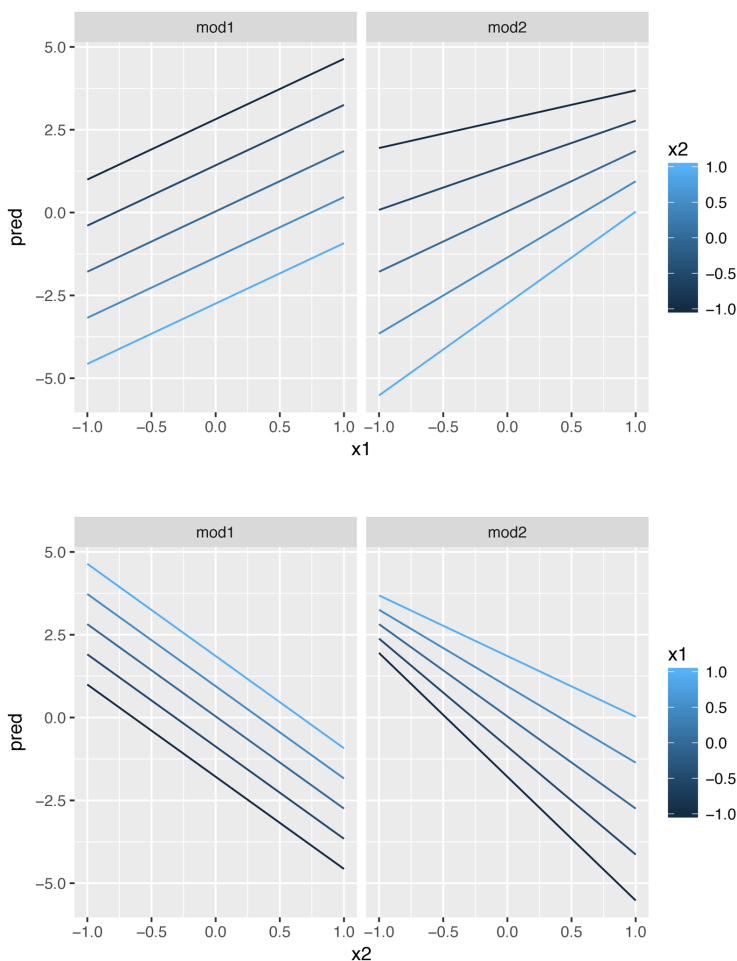
Next let's try and visualize that model. We have two continuous predictors, so you can imagine the model like a 3D surface. We could display that using `geom_tile()`:

```
ggplot(grid, aes(x1, x2)) +
  geom_tile(aes(fill = pred)) +
  facet_wrap(~ model)
```



That doesn't suggest that the models are very different! But that's partly an illusion: our eyes and brains are not very good at accurately comparing shades of color. Instead of looking at the surface from the top, we could look at it from either side, showing multiple slices:

```
ggplot(grid, aes(x1, pred, color = x2, group = x2)) +
  geom_line() +
  facet_wrap(~ model)
ggplot(grid, aes(x2, pred, color = x1, group = x1)) +
  geom_line() +
  facet_wrap(~ model)
```

This shows you that interaction between two continuous variables works basically the same way as for a categorical and continuous variable. An interaction says that there's not a fixed offset: you need to consider both values of x_1 and x_2 simultaneously in order to predict y .

You can see that even with just two continuous variables, coming up with good visualizations are hard. But that's reasonable: you shouldn't expect it will be easy to understand how three or more variables simultaneously interact! But again, we're saved a little because we're using models for exploration, and you can gradually

build up your model over time. The model doesn't have to be perfect, it just has to help you reveal a little more about your data.

I spent some time looking at the residuals to see if I could figure if `mod2` did better than `mod1`. I think it does, but it's pretty subtle. You'll have a chance to work on it in the exercises.

Transformations

You can also perform transformations inside the model formula. For example, $\log(y) \sim \sqrt{x_1} + x_2$ is transformed to $y = a_1 + a_2 * x_1 * \sqrt{x} + a_3 * x_2$. If your transformation involves $+$, $*$, $^$, or $-$, you'll need to wrap it in `I()` so R doesn't treat it like part of the model specification. For example, $y \sim x + I(x^2)$ is translated to $y = a_1 + a_2 * x + a_3 * x^2$. If you forget the `I()` and specify $y \sim x^2 + x$, R will compute $y \sim x * x + x$. $x * x$ means the interaction of x with itself, which is the same as x . R automatically drops redundant variables so $x + x$ becomes x , meaning that $y \sim x^2 + x$ specifies the function $y = a_1 + a_2 * x$. That's probably not what you intended!

Again, if you get confused about what your model is doing, you can always use `model_matrix()` to see exactly what equation `lm()` is fitting:

```
df <- tribble(
  ~y, ~x,
  1, 1,
  2, 2,
  3, 3
)
model_matrix(df, y ~ x^2 + x)
#> # A tibble: 3 × 2
#>   `(Intercept)`      x
#>   <dbl> <dbl>
#> 1         1         1
#> 2         1         2
#> 3         1         3
model_matrix(df, y ~ I(x^2) + x)
#> # A tibble: 3 × 3
#>   `(Intercept)` `I(x^2)`      x
#>   <dbl>      <dbl> <dbl>
#> 1         1         1         1
#> 2         1         4         2
#> 3         1         9         3
```

Transformations are useful because you can use them to approximate nonlinear functions. If you've taken a calculus class, you may have heard of Taylor's theorem, which says you can approximate any smooth function with an infinite sum of polynomials. That means you can use a linear function to get arbitrarily close to a smooth function by fitting an equation like $y = a_1 + a_2 * x + a_3 * x^2 + a_4 * x^3$. Typing that sequence by hand is tedious, so R provides a helper function, `poly()`:

```
model_matrix(df, y ~ poly(x, 2))
#> # A tibble: 3 × 3
#>   `(Intercept)` `poly(x, 2)1` `poly(x, 2)2`
#>   <dbl>         <dbl>         <dbl>
#> 1           1      -7.07e-01      0.408
#> 2           1      -7.85e-17     -0.816
#> 3           1       7.07e-01      0.408
```

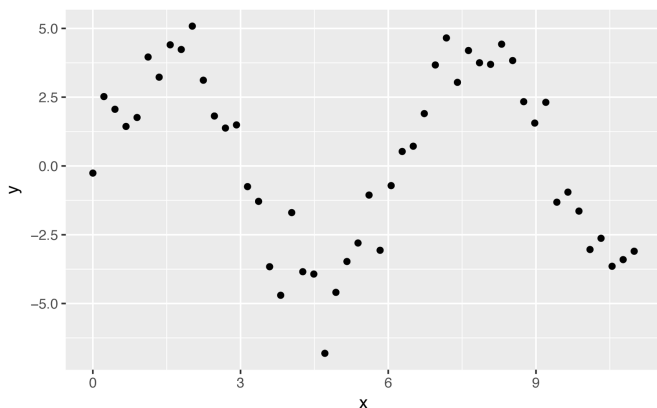
However there's one major problem with using `poly()`: outside the range of the data, polynomials rapidly shoot off to positive or negative infinity. One safer alternative is to use the natural spline, `splines::ns()`:

```
library(splines)
model_matrix(df, y ~ ns(x, 2))
#> # A tibble: 3 × 3
#>   `(Intercept)` `ns(x, 2)1` `ns(x, 2)2`
#>   <dbl>         <dbl>         <dbl>
#> 1           1      0.000      0.000
#> 2           1      0.566     -0.211
#> 3           1      0.344      0.771
```

Let's see what that looks like when we try and approximate a nonlinear function:

```
sim5 <- tibble(
  x = seq(0, 3.5 * pi, length = 50),
  y = 4 * sin(x) + rnorm(length(x))
)

ggplot(sim5, aes(x, y)) +
  geom_point()
```

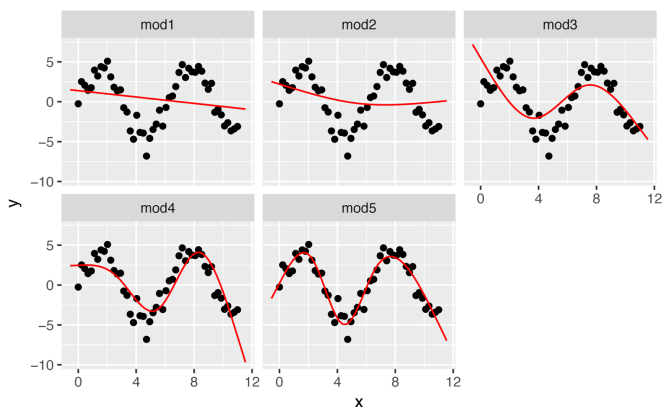


I'm going to fit five models to this data:

```
mod1 <- lm(y ~ ns(x, 1), data = sim5)
mod2 <- lm(y ~ ns(x, 2), data = sim5)
mod3 <- lm(y ~ ns(x, 3), data = sim5)
mod4 <- lm(y ~ ns(x, 4), data = sim5)
mod5 <- lm(y ~ ns(x, 5), data = sim5)

grid <- sim5 %>%
  data_grid(x = seq_range(x, n = 50, expand = 0.1)) %>%
  gather_predictions(mod1, mod2, mod3, mod4, mod5, .pred = "y")

ggplot(sim5, aes(x, y)) +
  geom_point() +
  geom_line(data = grid, color = "red") +
  facet_wrap(~ model)
```



Notice that the extrapolation outside the range of the data is clearly bad. This is the downside to approximating a function with a polynomial. But this is a very real problem with every model: the model can never tell you if the behavior is true when you start extrapolating outside the range of the data that you have seen. You must rely on theory and science.

Exercises

1. What happens if you repeat the analysis of `sim2` using a model without an intercept? What happens to the model equation? What happens to the predictions?
2. Use `model_matrix()` to explore the equations generated for the models I fit to `sim3` and `sim4`. Why is `*` a good shorthand for interaction?
3. Using the basic principles, convert the formulas in the following two models into functions. (Hint: start by converting the categorical variable into 0-1 variables.)

```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3)
```

4. For `sim4`, which of `mod1` and `mod2` is better? I think `mod2` does a slightly better job at removing patterns, but it's pretty subtle. Can you come up with a plot to support my claim?

Missing Values

Missing values obviously cannot convey any information about the relationship between the variables, so modeling functions will drop any rows that contain missing values. R's default behavior is to silently drop them, but `options(na.action = na.warn)` (run in the prerequisites), makes sure you get a warning:

```
df <- tribble(
  ~x, ~y,
  1, 2.2,
  2, NA,
  3, 3.5,
  4, 8.3,
  NA, 10
)
```

```
mod <- lm(y ~ x, data = df)
#> Warning: Dropping 2 rows with missing values
```

To suppress the warning, set `na.action = na.exclude`:

```
mod <- lm(y ~ x, data = df, na.action = na.exclude)
```

You can always see exactly how many observations were used with `nobs()`:

```
nobs(mod)
#> [1] 3
```

Other Model Families

This chapter has focused exclusively on the class of linear models, which assume a relationship of the form $y = a_1 * x_1 + a_2 * x_2 + \dots + a_n * x_n$. Linear models additionally assume that the residuals have a normal distribution, which we haven't talked about. There is a large set of model classes that extend the linear model in various interesting ways. Some of them are:

- *Generalized linear models*, e.g., `stats::glm()`. Linear models assume that the response is continuous and the error has a normal distribution. Generalized linear models extend linear models to include noncontinuous responses (e.g., binary data or counts). They work by defining a distance metric based on the statistical idea of likelihood.
- *Generalized additive models*, e.g., `mgcv::gam()`, extend generalized linear models to incorporate arbitrary smooth functions. That means you can write a formula like $y \sim s(x)$, which becomes an equation like $y = f(x)$, and let `gam()` estimate what that function is (subject to some smoothness constraints to make the problem tractable).
- *Penalized linear models*, e.g., `glmnet::glmnet()`, add a penalty term to the distance that penalizes complex models (as defined by the distance between the parameter vector and the origin). This tends to make models that generalize better to new datasets from the same population.
- *Robust linear models*, e.g., `MASS::rlm()`, tweak the distance to downweight points that are very far away. This makes them less sensitive to the presence of outliers, at the cost of being not quite as good when there are no outliers.

- *Trees*, e.g., `rpart::rpart()`, attack the problem in a completely different way than linear models. They fit a piece-wise constant model, splitting the data into progressively smaller and smaller pieces. Trees aren't terribly effective by themselves, but they are very powerful when used in aggregate by models like *random forests* (e.g., `randomForest::randomForest()`) or *gradient boosting machines* (e.g., `xgboost::xgboost.`)

These models all work similarly from a programming perspective. Once you've mastered linear models, you should find it easy to master the mechanics of these other model classes. Being a skilled modeler is a mixture of some good general principles and having a big toolbox of techniques. Now that you've learned some general tools and one useful class of models, you can go on and learn more classes from other sources.

Model Building

Introduction

In the previous chapter you learned how linear models worked, and learned some basic tools for understanding what a model is telling you about your data. The previous chapter focused on simulated datasets to help you learn about how models work. This chapter will focus on real data, showing you how you can progressively build up a model to aid your understanding of the data.

We will take advantage of the fact that you can think about a model partitioning your data into patterns and residuals. We'll find patterns with visualization, then make them concrete and precise with a model. We'll then repeat the process, but replace the old response variable with the residuals from the model. The goal is to transition from implicit knowledge in the data and your head to explicit knowledge in a quantitative model. This makes it easier to apply to new domains, and easier for others to use.

For very large and complex datasets this will be a lot of work. There are certainly alternative approaches—a more machine learning approach is simply to focus on the predictive ability of the model. These approaches tend to produce black boxes: the model does a really good job at generating predictions, but you don't know why. This is a totally reasonable approach, but it does make it hard to apply your real-world knowledge to the model. That, in turn, makes it difficult to assess whether or not the model will continue to work in the long term, as fundamentals change. For most real models, I'd

expect you to use some combination of this approach and a more classic automated approach.

It's a challenge to know when to stop. You need to figure out when your model is good enough, and when additional investment is unlikely to pay off. I particularly like this quote from reddit user Broseidon241:

A long time ago in art class, my teacher told me “An artist needs to know when a piece is done. You can't tweak something into perfection—wrap it up. If you don't like it, do it over again. Otherwise begin something new.” Later in life, I heard “A poor seamstress makes many mistakes. A good seamstress works hard to correct those mistakes. A great seamstress isn't afraid to throw out the garment and start over.”

—Broseidon241

Prerequisites

We'll use the same tools as in the previous chapter, but add in some real datasets: **diamonds** from **ggplot2**, and **flights** from **nycflights13**. We'll also need **lubridate** in order to work with the date/times in **flights**.

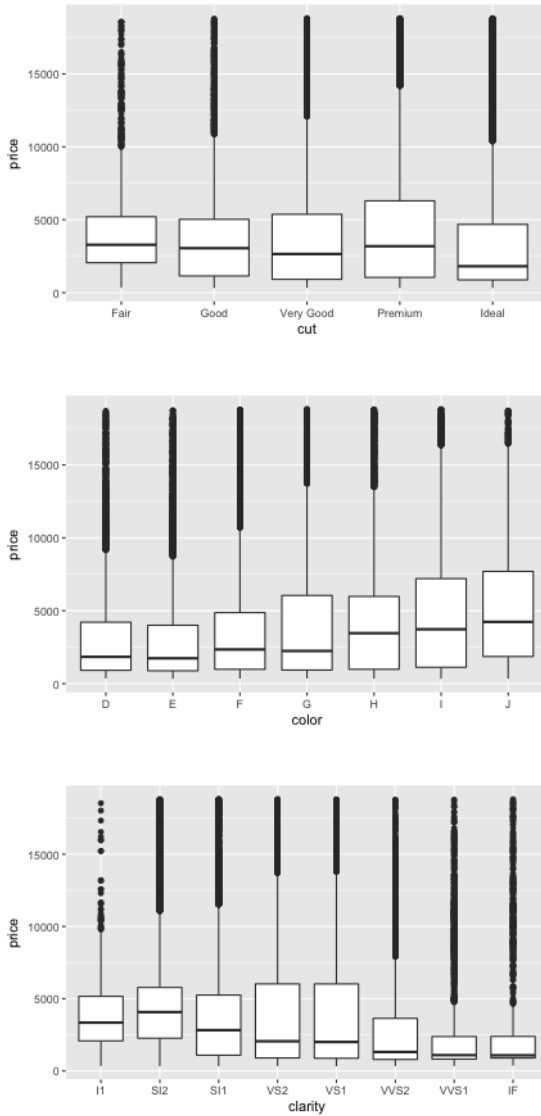
```
library(tidyverse)
library(modelr)
options(na.action = na.warn)

library(nycflights13)
library(lubridate)
```

Why Are Low-Quality Diamonds More Expensive?

In previous chapters we've seen a surprising relationship between the quality of diamonds and their price: low-quality diamonds (poor cuts, bad colors, and inferior clarity) have higher prices:

```
ggplot(diamonds, aes(cut, price)) + geom_boxplot()
ggplot(diamonds, aes(color, price)) + geom_boxplot()
ggplot(diamonds, aes(clarity, price)) + geom_boxplot()
```



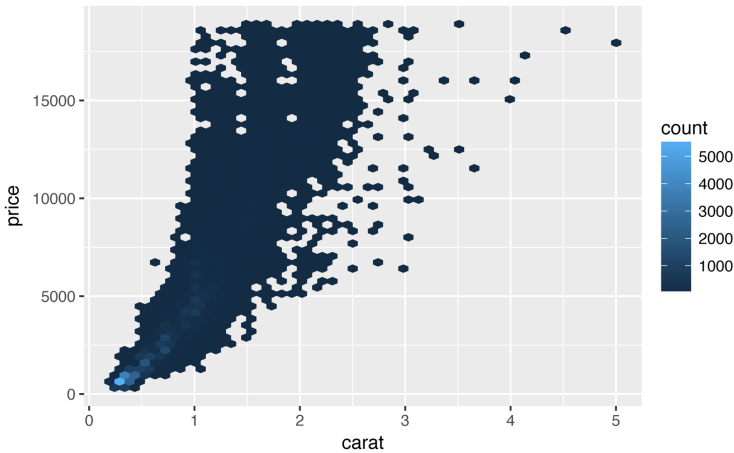
Note that the worst diamond color is J (slightly yellow), and the worst clarity is I1 (inclusions visible to the naked eye).

Price and Carat

It looks like lower-quality diamonds have higher prices because there is an important confounding variable: the weight (carat) of

the diamond. The weight of the diamond is the single most important factor for determining the price of the diamond, and lower-quality diamonds tend to be larger:

```
ggplot(diamonds, aes(carat, price)) +  
  geom_hex(bins = 50)
```



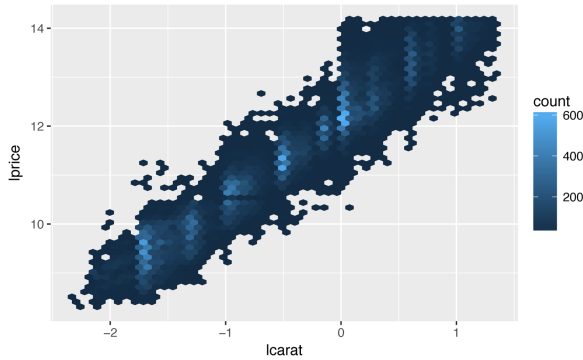
We can make it easier to see how the other attributes of a diamond affect its relative price by fitting a model to separate out the effect of `carat`. But first, let's make a couple of tweaks to the diamonds dataset to make it easier to work with:

1. Focus on diamonds smaller than 2.5 carats (99.7% of the data).
2. Log-transform the `carat` and `price` variables:

```
diamonds2 <- diamonds %>%  
  filter(carat <= 2.5) %>%  
  mutate(lprice = log2(price), lcarat = log2(carat))
```

Together, these changes make it easier to see the relationship between `carat` and `price`:

```
ggplot(diamonds2, aes(lcarat, lprice)) +  
  geom_hex(bins = 50)
```



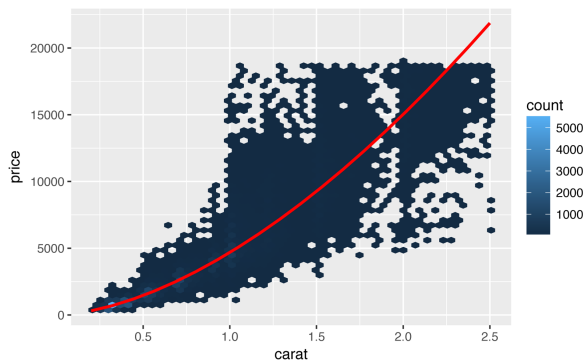
The log transformation is particularly useful here because it makes the pattern linear, and linear patterns are the easiest to work with. Let's take the next step and remove that strong linear pattern. We first make the pattern explicit by fitting a model:

```
mod_diamond <- lm(lprice ~ lcarat, data = diamonds2)
```

Then we look at what the model tells us about the data. Note that I back-transform the predictions, undoing the log transformation, so I can overlay the predictions on the raw data:

```
grid <- diamonds2 %>%
  data_grid(carat = seq_range(carat, 20)) %>%
  mutate(lcarat = log2(carat)) %>%
  add_predictions(mod_diamond, "lprice") %>%
  mutate(price = 2 ^ lprice)

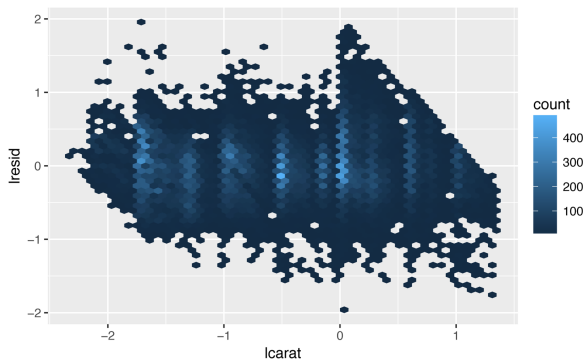
ggplot(diamonds2, aes(carat, price)) +
  geom_hex(bins = 50) +
  geom_line(data = grid, color = "red", size = 1)
```



That tells us something interesting about our data. If we believe our model, then the large diamonds are much cheaper than expected. This is probably because no diamond in this dataset costs more than \$19,000.

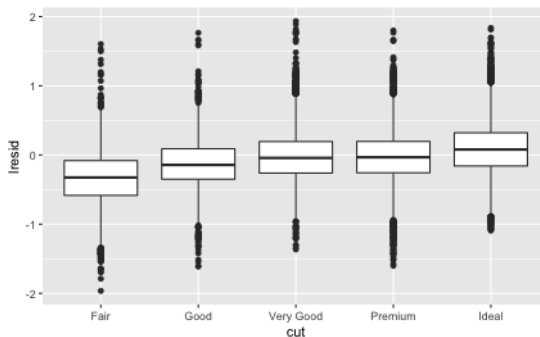
Now we can look at the residuals, which verifies that we've successfully removed the strong linear pattern:

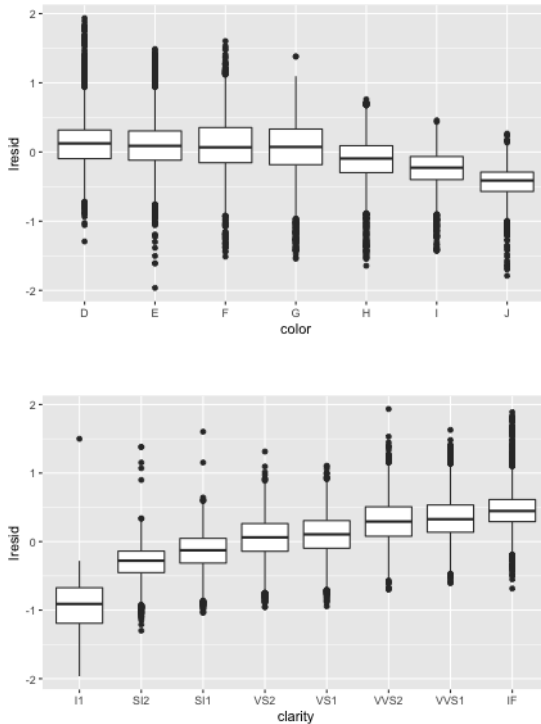
```
diamonds2 <- diamonds2 %>%  
  add_residuals(mod_diamond, "lresid")  
  
ggplot(diamonds2, aes(lcarat, lresid)) +  
  geom_hex(bins = 50)
```



Importantly, we can now redo our motivating plots using those residuals instead of price:

```
ggplot(diamonds2, aes(cut, lresid)) + geom_boxplot()  
ggplot(diamonds2, aes(color, lresid)) + geom_boxplot()  
ggplot(diamonds2, aes(clarity, lresid)) + geom_boxplot()
```





Now we see the relationship we expect: as the quality of the diamond increases, so to does its relative price. To interpret the y-axis, we need to think about what the residuals are telling us, and what scale they are on. A residual of -1 indicates that lprice was 1 unit lower than a prediction based solely on its weight. 2^{-1} is $1/2$, so points with a value of -1 are half the expected price, and residuals with value 1 are twice the predicted price.

A More Complicated Model

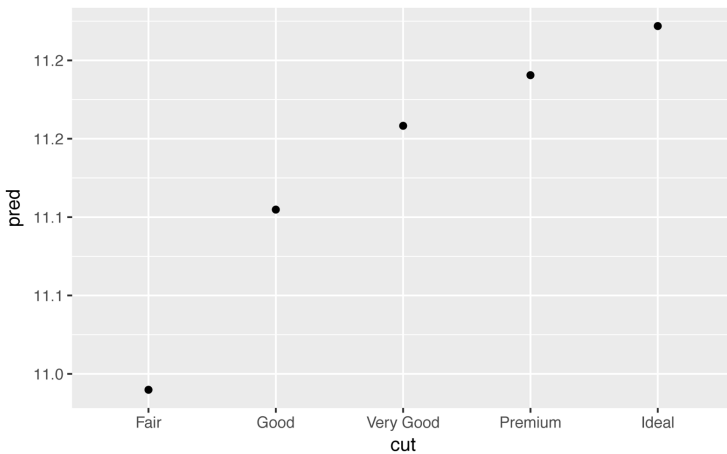
If we wanted to, we could continue to build up our model, moving the effects we've observed into the model to make them explicit. For example, we could include `color`, `cut`, and `clarity` into the model so that we also make explicit the effect of these three categorical variables:

```
mod_diamond2 <- lm(
  lprice ~ lcarat + color + cut + clarity,
  data = diamonds2
)
```

This model now includes four predictors, so it's getting harder to visualize. Fortunately, they're currently all independent, which means that we can plot them individually in four plots. To make the process a little easier, we're going to use the `.model` argument to `data_grid`:

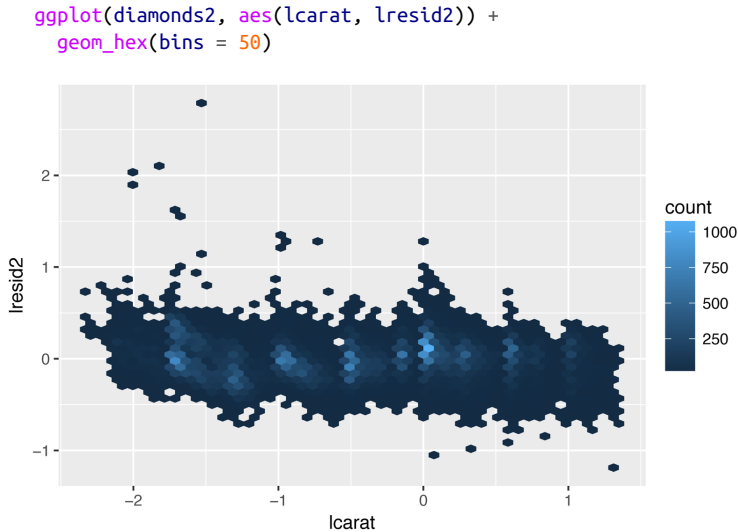
```
grid <- diamonds2 %>%
  data_grid(cut, .model = mod_diamond2) %>%
  add_predictions(mod_diamond2)
grid
#> # A tibble: 5 × 5
#>   cut lcarat color clarity pred
#>   <ord> <dbl> <chr>   <chr> <dbl>
#> 1 Fair -0.515 G SI1 11.0
#> 2 Good -0.515 G SI1 11.1
#> 3 Very Good -0.515 G SI1 11.2
#> 4 Premium -0.515 G SI1 11.2
#> 5 Ideal -0.515 G SI1 11.2

ggplot(grid, aes(cut, pred)) +
  geom_point()
```



If the model needs variables that you haven't explicitly supplied, `data_grid()` will automatically fill them in with the "typical" value. For continuous variables, it uses the median, and for categorical variables, it uses the most common value (or values, if there's a tie):

```
diamonds2 <- diamonds2 %>%
  add_residuals(mod_diamond2, "lresid2")
```

This plot indicates that there are some diamonds with quite large residuals—remember a residual of 2 indicates that the diamond is 4x the price that we expected. It’s often useful to look at unusual values individually:

```
diamonds2 %>%
  filter(abs(lresid2) > 1) %>%
  add_predictions(mod_diamond2) %>%
  mutate(pred = round(2 ^ pred)) %>%
  select(price, pred, carat:table, x:z) %>%
  arrange(price)
```

```
#> # A tibble: 16 × 11
#>   price  pred carat   cut color clarity depth table    x
#>   <int> <dbl> <dbl> <ord> <ord> <ord> <dbl> <dbl> <dbl>
#> 1  1013   264  0.25  Fair    F    SI2   54.4    64  4.30
#> 2  1186   284  0.25 Premium  G    SI2   59.0    60  5.33
#> 3  1186   284  0.25 Premium  G    SI2   58.8    60  5.33
#> 4  1262  2644  1.03  Fair    E    I1    78.2    54  5.72
#> 5  1415   639  0.35  Fair    G   VS2   65.9    54  5.57
#> 6  1415   639  0.35  Fair    G   VS2   65.9    54  5.57
#> # ... with 10 more rows, and 2 more variables: y <dbl>,
#> #   z <dbl>
```

Nothing really jumps out at me here, but it’s probably worth spending time considering if this indicates a problem with our model, or if there are errors in the data. If there are mistakes in the data, this could be an opportunity to buy diamonds that have been priced low incorrectly.

Exercises

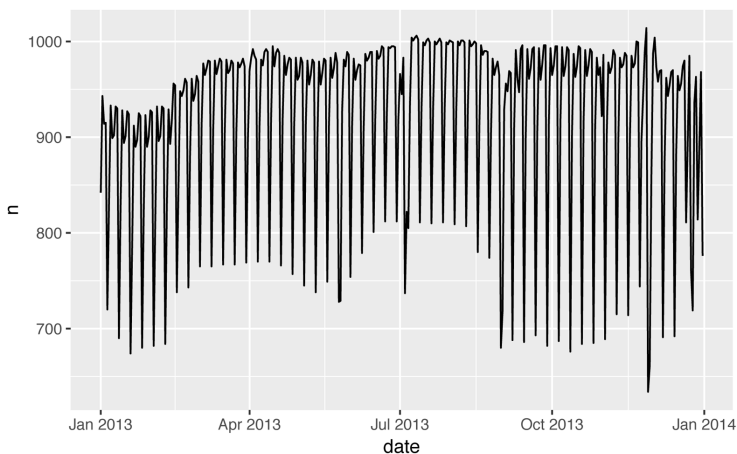
1. In the plot of `lcarat` versus `lprice`, there are some bright vertical strips. What do they represent?
2. If $\log(\text{price}) = a_0 + a_1 * \log(\text{carat})$, what does that say about the relationship between price and carat?
3. Extract the diamonds that have very high and very low residuals. Is there anything unusual about these diamonds? Are they particularly bad or good, or do you think these are pricing errors?
4. Does the final model, `mod_diamonds2`, do a good job of predicting diamond prices? Would you trust it to tell you how much to spend if you were buying a diamond?

What Affects the Number of Daily Flights?

Let's work through a similar process for a dataset that seems even simpler at first glance: the number of flights that leave NYC per day. This is a really small dataset—only 365 rows and 2 columns—and we're not going to end up with a fully realized model, but as you'll see, the steps along the way will help us better understand the data. Let's get started by counting the number of flights per day and visualizing it with **ggplot2**:

```
daily <- flights %>%
  mutate(date = make_date(year, month, day)) %>%
  group_by(date) %>%
  summarize(n = n())
daily
#> # A tibble: 365 × 2
#>   date      n
#>   <date> <int>
#> 1 2013-01-01  842
#> 2 2013-01-02  943
#> 3 2013-01-03  914
#> 4 2013-01-04  915
#> 5 2013-01-05  720
#> 6 2013-01-06  832
#> # ... with 359 more rows

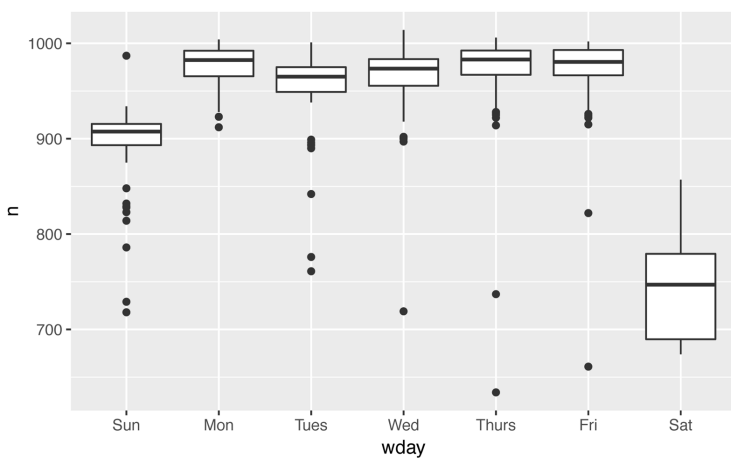
ggplot(daily, aes(date, n)) +
  geom_line()
```



Day of Week

Understanding the long-term trend is challenging because there's a very strong day-of-week effect that dominates the subtler patterns. Let's start by looking at the distribution of flight numbers by day of week:

```
daily <- daily %>%
  mutate(wday = wday(date, label = TRUE))
ggplot(daily, aes(wday, n)) +
  geom_boxplot()
```



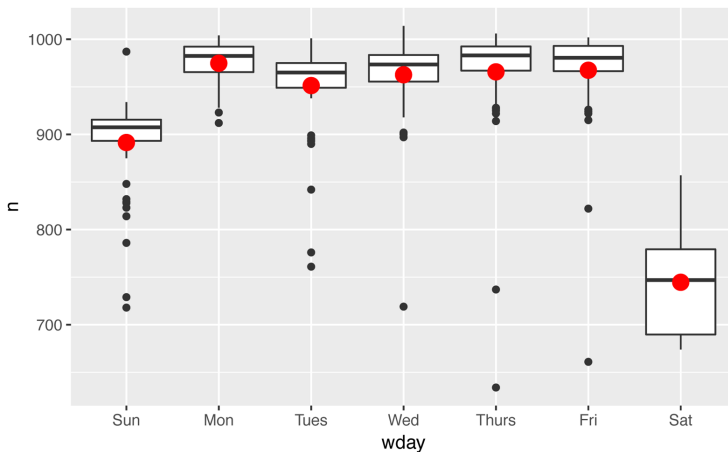
There are fewer flights on weekends because most travel is for business. The effect is particularly pronounced on Saturday: you might sometimes leave on Sunday for a Monday morning meeting, but it's very rare that you'd leave on Saturday as you'd much rather be at home with your family.

One way to remove this strong pattern is to use a model. First, we fit the model, and display its predictions overlaid on the original data:

```
mod <- lm(n ~ wday, data = daily)

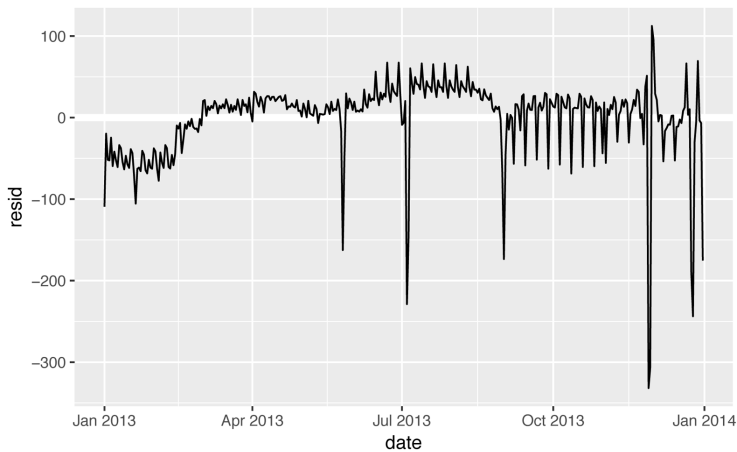
grid <- daily %>%
  data_grid(wday) %>%
  add_predictions(mod, "n")

ggplot(daily, aes(wday, n)) +
  geom_boxplot() +
  geom_point(data = grid, color = "red", size = 4)
```



Next we compute and visualize the residuals:

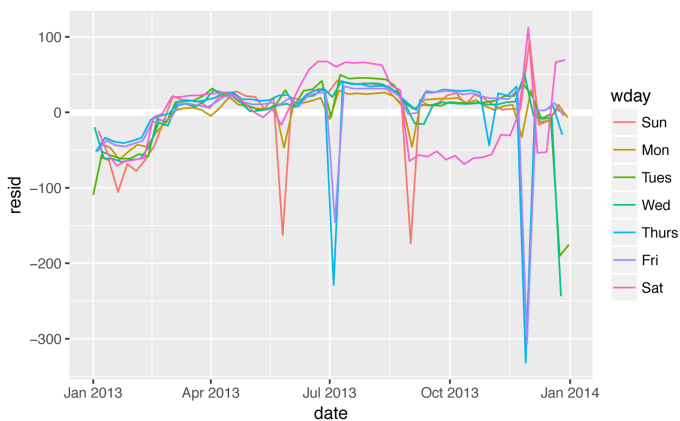
```
daily <- daily %>%
  add_residuals(mod)
daily %>%
  ggplot(aes(date, resid)) +
  geom_ref_line(h = 0) +
  geom_line()
```



Note the change in the y-axis: now we are seeing the deviation from the expected number of flights, given the day of week. This plot is useful because now that we've removed much of the large day-of-week effect, we can see some of the subtler patterns that remain:

- Our model seems to fail starting in June: you can still see a strong regular pattern that our model hasn't captured. Drawing a plot with one line for each day of the week makes the cause easier to see:

```
ggplot(daily, aes(date, resid, color = wday)) +  
  geom_ref_line(h = 0) +  
  geom_line()
```



Our model fails to accurately predict the number of flights on Saturday: during summer there are more flights than we expect, and during fall there are fewer. We'll see how we can do better to capture this pattern in the next section.

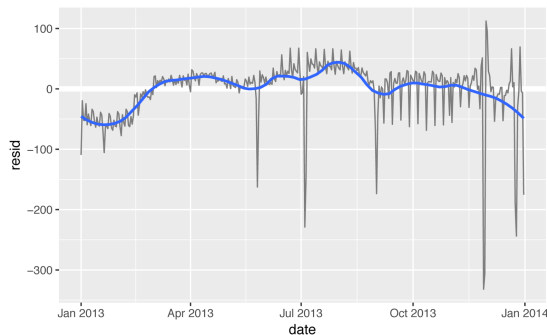
- There are some days with far fewer flights than expected:

```
daily %>%
  filter(resid < -100)
#> # A tibble: 11 × 4
#>   date      n wday resid
#>   <date> <int> <ord> <dbl>
#> 1 2013-01-01  842 Tues  -109
#> 2 2013-01-20  786 Sun   -105
#> 3 2013-05-26  729 Sun   -162
#> 4 2013-07-04  737 Thurs -229
#> 5 2013-07-05  822 Fri   -145
#> 6 2013-09-01  718 Sun   -173
#> # ... with 5 more rows
```

If you're familiar with American public holidays, you might spot New Year's Day, July 4th, Thanksgiving, and Christmas. There are some others that don't seem to correspond to public holidays. You'll work on those in one of the exercises.

- There seems to be some smoother long-term trend over the course of a year. We can highlight that trend with `geom_smooth()`:

```
daily %>%
  ggplot(aes(date, resid)) +
  geom_ref_line(h = 0) +
  geom_line(color = "grey50") +
  geom_smooth(se = FALSE, span = 0.20)
#> `geom_smooth()` using method = 'loess'
```

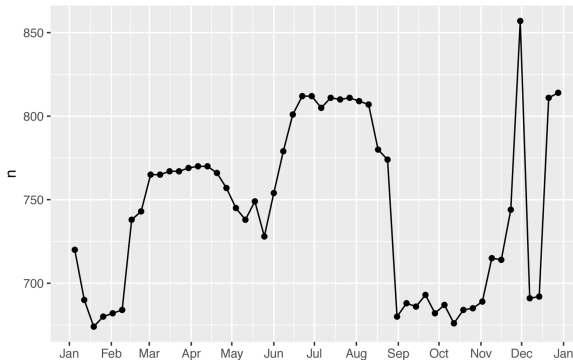


There are fewer flights in January (and December), and more in summer (May–Sep). We can't do much with this pattern quantitatively, because we only have a single year of data. But we can use our domain knowledge to brainstorm potential explanations.

Seasonal Saturday Effect

Let's first tackle our failure to accurately predict the number of flights on Saturday. A good place to start is to go back to the raw numbers, focusing on Saturdays:

```
daily %>%
  filter(wday == "Sat") %>%
  ggplot(aes(date, n)) +
    geom_point() +
    geom_line() +
    scale_x_date(
      NULL,
      date_breaks = "1 month",
      date_labels = "%b"
    )
)
```



(I've used both points and lines to make it more clear what is data and what is interpolation.)

I suspect this pattern is caused by summer holidays: many people go on holiday in the summer, and people don't mind travelling on Saturdays for vacation. Looking at this plot, we might guess that summer holidays are from early June to late August. That seems to line up fairly well with the **state's school terms**: summer break in 2013 was June 26–September 9.

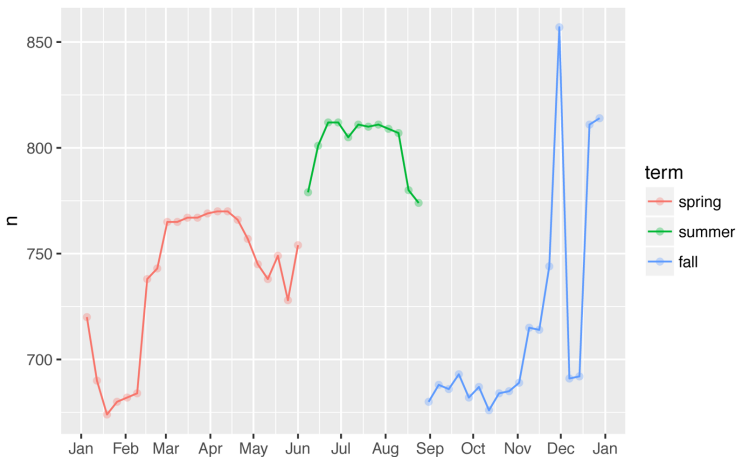
Why are there more Saturday flights in the spring than the fall? I asked some American friends and they suggested that it's less common to plan family vacations during the fall because of the big Thanksgiving and Christmas holidays. We don't have the data to know for sure, but it seems like a plausible working hypothesis.

Let's create a "term" variable that roughly captures the three school terms, and check our work with a plot:

```
term <- function(date) {
  cut(date,
      breaks = ymd(20130101, 20130605, 20130825, 20140101),
      labels = c("spring", "summer", "fall")
  )
}

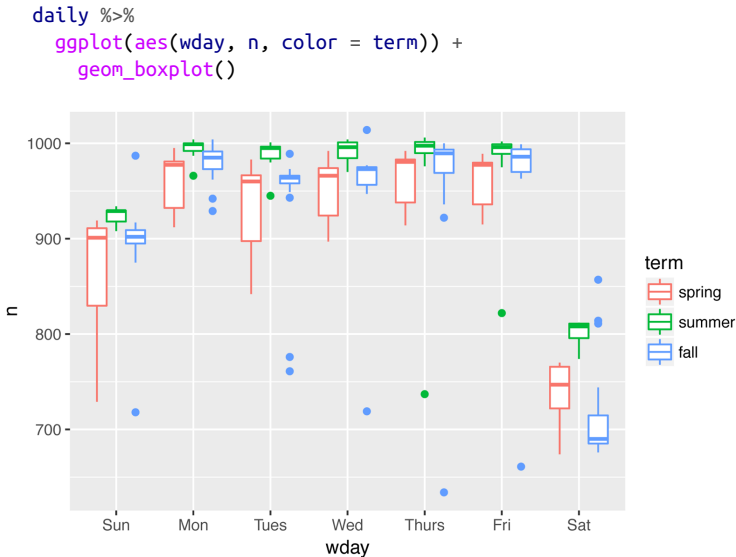
daily <- daily %>%
  mutate(term = term(date))

daily %>%
  filter(wday == "Sat") %>%
  ggplot(aes(date, n, color = term)) +
  geom_point(alpha = 1/3) +
  geom_line() +
  scale_x_date(
    NULL,
    date_breaks = "1 month",
    date_labels = "%b"
  )
)
```



(I manually tweaked the dates to get nice breaks in the plot. Using a visualization to help you understand what your function is doing is a really powerful and general technique.)

It's useful to see how this new variable affects the other days of the week:



It looks like there is significant variation across the terms, so fitting a separate day-of-week effect for each term is reasonable. This improves our model, but not as much as we might hope:

```
mod1 <- lm(n ~ wday, data = daily)
mod2 <- lm(n ~ wday * term, data = daily)

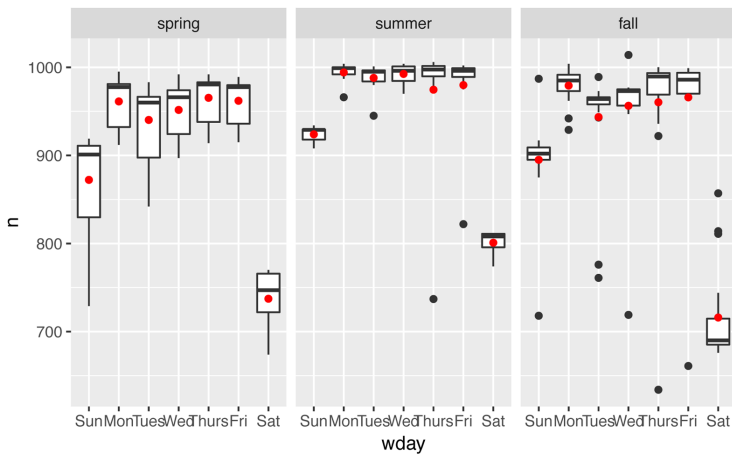
daily %>%
  gather_residuals(without_term = mod1, with_term = mod2) %>%
  ggplot(aes(date, resid, color = model)) +
  geom_line(alpha = 0.75)
```



We can see the problem by overlaying the predictions from the model onto the raw data:

```
grid <- daily %>%
  data_grid(wday, term) %>%
  add_predictions(mod2, "n")

ggplot(daily, aes(wday, n)) +
  geom_boxplot() +
  geom_point(data = grid, color = "red") +
  facet_wrap(~ term)
```

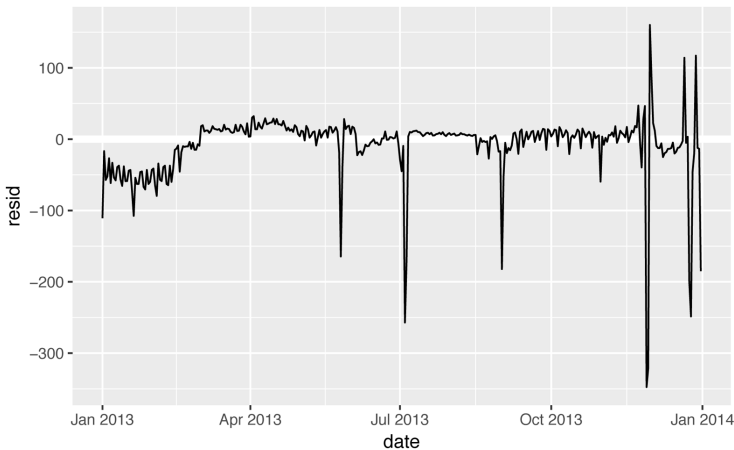


Our model is finding the *mean* effect, but we have a lot of big outliers, so the mean tends to be far away from the typical value. We can

alleviate this problem by using a model that is robust to the effect of outliers: `MASS::rlm()`. This greatly reduces the impact of the outliers on our estimates, and gives a model that does a good job of removing the day-of-week pattern:

```
mod3 <- MASS::rlm(n ~ wday * term, data = daily)

daily %>%
  add_residuals(mod3, "resid") %>%
  ggplot(aes(date, resid)) +
  geom_hline(yintercept = 0, size = 2, color = "white") +
  geom_line()
```



It's now much easier to see the long-term trend, and the positive and negative outliers.

Computed Variables

If you're experimenting with many models and many visualizations, it's a good idea to bundle the creation of variables up into a function so there's no chance of accidentally applying a different transformation in different places. For example, we could write:

```
compute_vars <- function(data) {
  data %>%
    mutate(
      term = term(date),
      wday = wday(date, label = TRUE)
    )
}
```

Another option is to put the transformations directly in the model formula:

```
wday2 <- function(x) wday(x, label = TRUE)
mod3 <- lm(n ~ wday2(date) * term(date), data = daily)
```

Either approach is reasonable. Making the transformed variable explicit is useful if you want to check your work, or use them in a visualization. But you can't easily use transformations (like splines) that return multiple columns. Including the transformations in the model function makes life a little easier when you're working with many different datasets because the model is self-contained.

Time of Year: An Alternative Approach

In the previous section we used our domain knowledge (how the US school term affects travel) to improve the model. An alternative to making our knowledge explicit in the model is to give the data more room to speak. We could use a more flexible model and allow that to capture the pattern we're interested in. A simple linear trend isn't adequate, so we could try using a natural spline to fit a smooth curve across the year:

```
library(splines)
mod <- MASS::rlm(n ~ wday * ns(date, 5), data = daily)
```

```
daily %>%
  data_grid(wday, date = seq_range(date, n = 13)) %>%
  add_predictions(mod) %>%
  ggplot(aes(date, pred, color = wday)) +
  geom_line() +
  geom_point()
```

